

Meta-Attention Architectures

Make attention itself learn how to attend optimally under different tasks or constraints.

Most labs train attention weights — this phase trains attention to learn how to compute its own weights. By introducing hypernetworks that generate Q/K/V parameters conditioned on the input distribution, you build a mechanism that adapts structurally to each task rather than merely numerically.

CORE TASKS

- Design an attention mechanism that adapts its own Q/K/V computation dynamically based on the input distribution.
- Introduce attention hypernetworks: small networks that generate attention parameters per sequence.
- Evaluate task transfer: can the same learned meta-attention generalize across multiple synthetic tasks?
- Track how meta-parameters evolve during training using custom dashboards.

MATHEMATICAL FORMULATION

```
Q = HN(x) · W_Q, K = HN(x) · W_K, V = HN(x) · W_V
where HN(x) = hypernetwork conditioned on input distribution x
Meta-loss: L_meta = SUM_t L_t(theta, phi_t) [joint over tasks t, params phi]
```

INSTRUMENTATION

- Track meta-parameter evolution over training steps and across random seeds.
- Analyze gradient alignment between tasks at each layer.
- Measure generalization gap: within-task vs. cross-task loss surfaces.
- Visualize hypernetwork weight distributions as training progresses.

DELIVERABLES

Type	Description
Code	Hypernetwork attention module (PyTorch/JAX) with modular task adapters
Analysis	Gradient alignment report across 3+ synthetic tasks
Writeup	4-page paper-style report: hypothesis → result → interpretation → open questions

Viz	Training dashboard showing meta-parameter trajectory over steps
-----	---

RESEARCH INSIGHT

Few labs explicitly train attention to learn how to learn attention. This is meta-learning at the architectural level — beyond standard MAML-style approaches by operating on the attention computation itself.

Difficulty: Advanced | Meta-learning · Hypernetworks · Transfer